

08. Proxy aur Reflect

Proxy kya hai?

Kisi object ke upar ek **wrapper** jo uske operations (read, write, delete, etc.) ko **beech me pakad** (intercept) sakta hai.

```
js
let proxy = new Proxy(target, handler);
```

- `target`: jise wrap karna hai (function bhi ho sakta hai)
- `handler`: object jisme **traps** (methods) hote hain
- Trap nahi diya to operation seedha `target` par chala jaata hai.

Common Traps

Trap	Kab chalta hai
<code>get</code>	property read
<code>set</code>	property write
<code>has</code>	<code>in</code> operator
<code>deleteProperty</code>	<code>delete</code>
<code>apply</code>	function call
<code>construct</code>	<code>new</code>
<code>ownKeys</code>	<code>Object.keys</code> , <code>for..in</code> etc.

Examples

Default value (`get`):

```
js
numbers = new Proxy([0, 1, 2], {
  get(target, prop) { return prop in target ? target[prop] : 0; }
});
numbers[123]; // 0
```

Validation (`set`): sirf number allow:

```
js
numbers = new Proxy([], {
  set(target, prop, val) {
    if (typeof val == 'number') { target[prop] = val; return true; }
    return false; // TypeError
  }
});
```

`set` me `true` return karna zaruri hai, warna error aata hai.

Private jaisi properties (`__` se shuru): `get`, `set`, `deleteProperty`, `ownKeys` traps se chhupa sakte ho.

Range check (`has`):

```
js
range = new Proxy({start: 1, end: 10}, {
  has(t, p) { return p >= t.start && p <= t.end; }
});
5 in range; // true
```

Function wrap (`apply`): `delay(f, ms)` decorator proxy se banao to `f.length`, `f.name` jaisi properties bhi bani rehti hain.

Reflect

Ek built-in object jiske methods proxy traps ke **same naam aur arguments** ke saath hote hain. Operation ko original object tak forward karne ke liye use hota hai:

```
js
get(target, prop, receiver) {
  console.log(`GET ${prop}`);
  return Reflect.get(target, prop, receiver);
}
```

`receiver` (teesra argument) getters ke saath sahi `this` dene ke liye zaruri hai (jaise inheritance me). Isliye `target[prop]` ke bajay `Reflect.get(...)` behtar hai.

Limitations

- Internal slots** wale built-ins (`Map`, `Set`, `Date`, `Promise`) proxy karne par methods fail ho sakte hain. Fix: `get` trap me function ko `value.bind(target)` karo. (`Array` me ye problem nahi.)

- **Private class fields** (`#x`) ke saath bhi yehi problem.
- `proxy !== target`, aur `===` ko intercept nahi kar sakte.
- Performance thodi slow hoti hai.

Revocable Proxy

Baad me band karne layak proxy:

js

```
let { proxy, revoke } = Proxy.revocable(obj, {});  
revoke();          // ab proxy.data par error
```

Kahan use hota hai

Default values, validation, observable objects (change detect karna), access control, function decorators.